

categorical_crossentropy. It is used to optimize classification models. Some of the available choices for optimizer include SGD, RMSprop, Adagrad, Adadelta, Adam, Adamax, Nadam, etc. In our model, we have used an optimizer called 'adam' (Adam). Adam is a first-order gradient-based optimization algorithm for stochastic objective functions. A detailed discussion of Adam is beyond the scope of our tutorial. Finally, a metric is a function that is used to measure the performance of a model. In our model, we have given 'accuracy' as the metric. This metric calculates how often predictions equal labels. The inclusion of this metric is the reason why we were able to print the accuracy of our model while processing the test data. In addition to the accuracy metric, there are other metrics like probabilistic metric, regression metric, image segmentation metric, etc.

Now, consider the 20th line of code shown below, used in developing our model.

```
model.fit(x_train, y_train, batch_
size=batch_size, epochs=epochs,
validation_split=0.1)
```

Let us try to understand the above line of code. The following parameters are passed to the *fit()* method. The variable *x_train* contains the training data. Training data could be either a NumPy array or a TensorFlow tensor. The variable *y_train* contains the target data. Target data could also be either a NumPy array or a TensorFlow tensor. Parameters to specify the batch size and the number of epochs are also passed to the *fit()* method. The parameter *validation_split* fixes the fraction of the training data to be used as validation data. Thus, the parameter takes a floating point value between 0 and 1. In our model, the value specified for this parameter is 0.1. After training, this model is saved in our system so that unnecessary

training can be avoided in the future. Later, the test data is used to test the accuracy of the model we have trained. Finally, our model can be used to make predictions about new images of handwritten digits. With this, the discussion about our first trained model is complete.

Unsupervised learning with scikit-learn

We know that the two important paradigms of machine learning are supervised learning and unsupervised learning. Though not as popular as these two models, there are other paradigms also, like semi-supervised learning and reinforcement learning. But our discussion so far in this series was exclusively based on supervised and unsupervised learning. Though we have discussed unsupervised learning, we have given a lot of importance to supervised learning algorithms like linear regression and classification so far.

Now, it is time to discuss more about unsupervised learning, which works with unlabelled data. Just like supervised learning, in unsupervised learning also there are different algorithms like clustering, dimensionality reduction, etc. In this article, we will be focusing on clustering. Clustering is grouping a set of objects in such a way that objects in the same group are more related to each other than objects in other groups. There are many different clustering algorithms — K-Means clustering, affinity propagation clustering, mean shift clustering, spectral clustering, hierarchical clustering, DBSCAN clustering, OPTICS clustering, etc. In this article, we will be implementing K-Means clustering using the library scikit-learn.

The K-Means clustering algorithm begins by choosing K random points as cluster centres. Objects to be grouped are assigned to one of these K randomly chosen clusters. Based on the number of objects assigned

to each of the clusters, new cluster centres are repeatedly calculated until the grouping of objects becomes stable. Now, let us get familiar with the Python script, shown below, implementing K-Means clustering using scikit-learn. Since the program is relatively large, a part by part explanation is given here for better understanding. Line numbers are added to the program for ease of explanation.

```
1. import numpy as np
2. from matplotlib import pyplot as plt
3. from sklearn.datasets import make_
   blobs
4. from sklearn.cluster import KMeans
5. X, y = make_blobs(n_samples=200,
   centers=2, n_features=2, cluster_
   std=0.5, random_state=0)
6. plt.scatter(X[:,0], X[:,1])
7. plt.show( )
```

Let us try to understand the above lines of code. Lines 1 and 2 import NumPy and Matplotlib. Line 3 imports the method *make_blobs()*. Line 4 imports the class *KMeans()*. Line 5 calls the method *make_blobs()* to generate isotropic Gaussian blobs for clustering. A collection of binary data stored as a single entity is called a blob (binary large object). But what do I mean by the previous two sentences? Well, in this example, we are not using actual data; instead, we are generating sample data to test the K-Means clustering algorithm. But unlike many other occasions in this series, we are not generating random data this time. The sample data generated depends on the parameters we pass to the method *make_blob()*. In this example, we are taking 200 samples (parameter *n_samples*) with just two centres (parameter *centers*). The parameter *n_features* determines the number of columns or features the generated data set will have. In this case it is 2. Since we are plotting the generated data in a two-dimensional plane, *n_features* should be at least 2. The parameter *cluster_std* fixes the standard deviation of the clusters. In this case